

1)

week - 2.

A. Soniya

1) A variable is a named container used to store data in memory. The stored value can be used, updated or manipulated later in program.

Eg); let name = "Rahul";

⇒ name = variable

Rahul = value

2) An identifier is the name given to a variable, function, class, or object. A variable's value is the actual data stored in that value.

Eg) let student name = "Ravi";

⇒ studentname = Identifier

Ravi = value

3) A ~~Java~~ Valid Javascript identifier must follow these rules:

1) Must start with a letter, underscore (_), or dollar sign (\$).

2) Cannot start with a number

3) Cannot contain spaces

4) Cannot use Javascript reserved keywords.

5) Javascript identifiers are case-sensitive.

4) keyword

Redeclare

Reassign

Scope

var

Yes

Yes

Function scope

let

No

Yes

Block scope

const

No

No

Block scope

5) A const variable cannot be reassigned after it is declared. However, if a const variable stores an object or array, its properties or elements can still be modified.

because `const` protects the reference, not the content.

6) Hoisting is JavaScript's behavior of moving declarations (not initialization) to the top of their scope before executing code. The code.

7) The Temporal Dead Zone (TDZ) is the time between entering a block and initializing a `let` or `const` variable. During this period, accessing the variable causes a reference error.

8). `var` is hoisted and initialized with `undefined`.

`let` and `const` are also hoisted but remain in the Temporal Dead Zone.

9) Scope determines where a variable can be accessed in a program. JavaScript has three main types of scope.

1) Global scope

2) Function scope

3) Block scope.

10) `var` is discouraged because it is function-scoped, allows redeclaration, and its hoisting behavior can lead to unexpected bugs. Modern JavaScript recommends using `let` & `const` for safe and more predictable code.

II)

11) 10

12) 2

13) `undefined`. Because `var` is hoisted & initialized with `undefined`.

14) Reference Error

15) kumar

16) undefined

17) 11

5 $\Rightarrow$ local variables hides the global variable

18) 3

19) Reference Error $\Rightarrow$ Var is Accessible and let is not accessible

20) [1, 2, 3, 4]

VI:

21) Type Error occurs because a const variable cannot be reassigned after initialization.

22) A Syntax Error occurs because let is a reserved keyword and cannot be used as an identifier.

23) A Syntax Error, occurs because Javascript identifiers cannot begin with a number.

24) A reference Error occurs because let variables cannot be accessed before initialization due to the temporal Dead zone.

25) A Syntax Error occurs because a const declaration requires an initial value.

26) var declarations are hoisted and initialized with undefined, so the variable exists before assignment.

27) A syntax Error occurs because let does not allow redeclaration within the same scope.

28) A Type Error occurs because a const variable cannot be assigned a new value.

29) A reference Error occurs because let variables are block-scoped and cannot be accessed outside the block where they are declared.

30) No error. Automatic semicolon insertion (ASI) makes it valid.

IV.

30) let name = "soniya";

console.log

31) let username = "soniya";

console.log(username);

O/P

soniya

32) const BIRTH_YEAR = 2004;

let age = 2026 - BIRTH_YEAR;

console.log(age);

O/P

22

33) let a = 10;

let b = 20;

[a, b] = [b, a]

console.log("a =", a);

console.log("b =", b);

O/P

a = 20

b = 10

```
34) var name = "soniya";  
    let age = 22;  
    const isStudent = true;  
  
    console.log(typeof name);  
    console.log(typeof age);  
    console.log(typeof isStudent);
```

O/P:

String
number
Boolean

```
35) console.log(city);  
    let city = 'chennai';
```

O/P:

ReferenceError:

cannot access 'city' before initialization

```
36) {  
    let message = "Hello"  
    console.log(message);  
}  
console.log(message);
```

O/P:

Hello

ReferenceError: message is not defined

```
37) const student = {  
    name: "soniya",  
    age: 22  
};
```

student.age = 23;

console.log(student);

O/P

```
{  
  name: "Soniya"  
  age: 23  
}
```

38)

```
function demo () {  
  var x = 10;  
  var x = 20;  
  console.log(x);  
}  
demo();
```

// var allows redeclaration within the same function scope.

O/P: 20.

39)

```
let name = "Global"
```

```
function show () {
```

```
  let name = "Local";
```

```
  console.log(name);
```

```
}
```

```
show();
```

```
console.log(name);
```

O/P:

Local

Global

40)

```
let functions = [];
```

```
for (let i = 0; i < 3; i++) {
```

```
  functions.push(function () {
```

```
    console.log(i);
```

```
  });
```

```

}
functions [0] ();
functions [1] ();
functions [2] ();

```

O/P:

```

0
1
2

```

V).

41) let.

A bank account balance changes whenever money is deposited or withdrawn. Since the value changes during the program let is used.

Eg) let accountBalance = 5000;
 accountBalance += 1000;
 console.log(accountBalance);

42) const

India's independence year (1947) never changes, so it should be stored as a const.

Eg). const INDEPENDENCE_YEAR = 1947;
 console.log(INDEPENDENCE_YEAR);

43) let

The cart total changes whenever items are added or removed.

Eg) let cartTotal = 1500;
 cartTotal += 500;
 console.log(cartTotal);

44) Const

The configuration value remains fixed throughout the applications execution.

Eg) `const MAX-LOGIN-ATTEMPTS = 5;`

`console.log(MAX-LOGIN-ATTEMPTS);`

45) let

The login status changes during the user's session.

Eg) `let isLoggedIn = false;`

`isLoggedIn = true`

`console.log(isLoggedIn);`

46). Const

A product ID is fixed for that product and does not change after the page loads.

Eg). `const PRODUCT-ID = 102`

`console.log(PRODUCT-ID);`

47). Var is function scoped, so all event handlers share the same loop variable. After the loop finishes every handler refers to the final value of `i`.

let is block scoped, so each iteration creates a new `i`. Therefore every button remembers its own index correctly.

Eg) `for (var i = 0; i < 3; i++) {`

`set Timeout(() => console.log(i), 0);`

o/p
3
3
3

```

41) for (let i = 0; i < 3; i++) {
    setTimeout(() => console.log(i), 0);
}

```

O/P
0
1
2

48) let
The temperature value changes every second, so it must be reassignable.

```

Eg) let temperature = 30;
    temperature = 31;
    console.log(temperature);

```

49) const
An API key should never change while the application is running. Declaring it with const prevents accidental reassignment and improves code safety and readability.

```

Eg) const API_KEY = "abc123xyz";
    console.log(API_KEY);

```

50) Base salary => const

Bonus salary => let

The base salary remains fixed for the year. whereas the monthly bonus changes every month.

```

Eg) const BASE_SALARY = 50000;
    let monthlyBonus = 5000;

```

```

    monthlyBonus = 8000;
    console.log(BASE_SALARY);
    console.log(monthlyBonus);

```

O/P
50000
8000